MASTER 1 SIGLIS · EXPOSÉ DEVOPS - DEPENDABOT



Illustration : logo de Dependabot, © GitHub

Dependabot

Automatiser la mise à jour sécurisée des dépendances sur GitHub

SYNTHÈSE TECHNIQUE

1 · Détecter → **2 · Proposer** → **3 · Vérifier** → **4 · Décider**

RÉALISÉ PAR (BINÔME)

Madou DIAKAYETEBA
Mouhamed S. CAMARA

SOUS LA DIRECTION DE

Pr . Bourcier

Octobre 2026 · Dépôt de démonstration : github.com

Plan

Cette synthèse part du problème que Dependabot résout, explique son fonctionnement et sa configuration, puis situe son rôle exact, et ses limites, dans un pipeline CI/CD. Chaque point est illustré par notre dépôt de démonstration, parti volontairement d'une version ancienne d'Axios.

Introduction **3**

1. Dependabot et le problème qu'il résout **3**

1.1 Un robot de mise à jour intégré à GitHub 3

1.2 Des dépendances qui vieillissent mal 3

2. Fonctionnement et configuration **4**

2.1 Repérer une dépendance vulnérable 4

2.2 Deux types de mises à jour 4

2.3 Le fichier dependabot.yml 5

3. Dependabot dans un pipeline CI/CD **6**

3.1 Détecter, proposer, vérifier, décider 6

3.2 Limites et points de vigilance 6

Conclusion **7**

Sources **7**

FIL CONDUCTEUR

Dependabot **détecte** et **propose** ; le pipeline CI **vérifie** ; l'équipe **décide**.

Introduction

Une application moderne est surtout faite de code écrit par d'autres. Notre application de démonstration (React, Vite, TypeScript) déclare **15 dépendances** dans son package.json, mais son lockfile en fige **210** : chaque bibliothèque apporte les siennes (dépendances indirectes, dites « transitives »). Suivre à la main les failles et les nouvelles versions de chacune est irréaliste. **Dependabot**, le robot intégré à GitHub, automatise cette veille : il repère les dépendances vulnérables ou dépassées et propose de les mettre à jour sous forme de *pull requests*.

Cette synthèse présente ce qu'il est et le problème qu'il résout (1), son fonctionnement et sa configuration (2), puis son rôle exact et ses limites dans un pipeline CI/CD (3), à partir de notre dépôt de démonstration, parti volontairement d'axios 0.18.0.

1. Dependabot et le problème qu'il résout

1.1 Un robot de mise à jour intégré à GitHub

Dependabot lit les fichiers qui déclarent les dépendances d'un projet, le **manifeste** (package.json) et le **lockfile** qui fige les versions exactes installées (package-lock.json), repère celles qui sont vulnérables ou dépassées, puis ouvre de lui-même une **pull request** (PR) contenant la mise à jour. Service indépendant racheté par GitHub en mai 2019, il est aujourd'hui intégré et gratuit : rien à installer, il s'active dans *Settings › Advanced Security* et se règle par un fichier YAML versionné avec le code, `.github/dependabot.yml`. Il couvre une trentaine d'écosystèmes (npm, pip, Maven, Gradle, Go modules, Cargo, NuGet, Docker, GitHub Actions, Terraform…). Sous ce nom se cachent trois fonctions :

Fonction	Ce qu'elle fait	Configuration
Alerts	Signale chaque dépendance touchée par une faille connue, avec sa gravité et la version qui corrige	Aucune (activation dans Settings)
Security updates	Ouvre une PR vers une version qui corrige la faille	Aucune (activation dans Settings)
Version updates	Ouvre régulièrement des PR vers les nouvelles versions, même sans faille	Fichier dependabot.yml obligatoire

Tableau 1 : les trois fonctions de Dependabot.

1.2 Des dépendances qui vieillissent mal

Une faille est souvent découverte des années après la publication d'une version. Publiée le 19 février 2018, axios 0.18.0 a reçu sa première faille en mai 2019 (CVE-2019-10742, déni de service) ; à l'activation de Dependabot sur notre dépôt, le 28 septembre 2026, **25 alertes** la concernaient, dont **11 de gravité élevée**. Le code n'a pas changé ; c'est le risque qui a augmenté. Avec CVE-2025-27152 par exemple, un client axios créé avec une adresse de base (baseURL) et une clé d'API secrète envoie la requête, clé comprise, vers un serveur tiers si on lui passe une adresse absolue fournie par un utilisateur.

Gérées à la main, les mises à jour reposent sur la bonne volonté de chacun, et « ça marche, on n'y touche pas » devient la règle. Le cas d'école est **Equifax** : le correctif de la faille Apache Struts CVE-2017-5638, disponible depuis le 7 mars 2017, n'a pas été appliqué sur le portail concerné, exploité de mai à juillet 2017 ; les données d'environ 147 millions de personnes ont été exposées. C'est l'enjeu de la **supply chain security** (sécurité de la chaîne d'approvisionnement logicielle) : s'assurer que les « pièces » d'une application ne sont ni **défectueuses** (failles connues), ni **piégées** (versions malveillantes). Dependabot traite d'abord le premier risque ; son délai d'attente (*cooldown*, section 3.2) réduit le second.

2. Fonctionnement et configuration

2.1 Repérer une dépendance vulnérable

1. **Le graphe de dépendances.** GitHub analyse manifestes et lockfiles et en déduit la liste exacte des dépendances et de leurs versions, transitives comprises.
2. **La GitHub Advisory Database.** Base publique des failles connues (identifiants CVE et GHSA) ; seuls les avis relus par GitHub déclenchent des alertes.
3. **La comparaison.** Si la version installée tombe dans une plage vulnérable (axios « < 0.21.2 » pour CVE-2021-3749), une alerte est créée avec la gravité et la première version corrigée. Elle est refaite à chaque nouvel avis et à chaque push qui modifie les dépendances.

Pour les versions updates, le principe est plus simple : à chaque échéance du calendrier, Dependabot compare la version utilisée à la dernière publiée sur le registre (npmjs.org pour npm).

DANS NOTRE DÉPÔT

37 alertes entre le 28 et le 30 septembre 2026 : axios (27), lodash (6), vite (3) et esbuild (1), une dépendance transitive de vite 5. Toutes sont aujourd'hui fermées (état *Fixed*), après la fusion des PR n° 9, 10 et 11.

2.2 Deux types de mises à jour

Dependabot ouvre deux sortes de PR, comparées dans le tableau 2. Les **security updates** répondent à une alerte : elles sont ouvertes immédiatement, échappent à la limite de PR et visent une version qui corrige la faille, pas forcément la dernière. Les **version updates** suivent le calendrier fixé dans dependabot.yml (chez nous, daily) et proposent la dernière version publiée, dans la limite de 5 PR ouvertes par défaut.

	Security updates	Version updates
Déclencheur	Une alerte (faille connue)	Le calendrier (schedule, chez nous daily)
Configuration	Activation dans Settings ; dependabot.yml facultatif	dependabot.yml obligatoire
Version proposée	Une version qui corrige la faille (axios : 0.33.0, pas 1.20.0)	La dernière version publiée
Limite de PR	Aucune	5 PR ouvertes au maximum (valeur par défaut, la nôtre)
Délai	Immédiat	3 jours après la publication (*cooldown* par défaut depuis juillet 2026)
Dans notre dépôt	n° 9 : axios 0.18.0 → 0.33.0 n° 10 : vite 5.4.21 → 8.3.1 n° 11 : axios 0.34.0 + lodash 4.18.1 (groupée)	n° 8 : axios 0.34.0 → 1.20.0 n° 4 à 7 : typescript, eslint-plugin-react-refresh, @types/node, globals

Tableau 2 : security updates et version updates, avec les PR de notre dépôt.

Observé dans notre dépôt. Le 28 septembre 2026, quelques minutes après le premier push, 29 alertes étaient ouvertes et Dependabot a ouvert 10 PR en deux minutes : 2 de sécurité (n° 9, axios 0.18.0 → 0.33.0, et n° 10, vite 5.4.21 → 8.3.1) et 8 de version (n° 1 à 8). Le 30 septembre, deux nouveaux avis visant axios < 0.34.0 ont été publiés : Dependabot a aussitôt ouvert la PR de sécurité **groupée** n° 11 (axios 0.33.0 → 0.34.0 et lodash 4.17.15 → 4.18.1), effet de l'option *Grouped security updates* des paramètres, puis fermé la PR de version lodash n° 3, devenue inutile. Ces traitements tournent sur des runners GitHub Actions (onglet *Actions*, « Dependabot Updates »).

Chaque PR est prête à relire : titre explicite (build(deps): bump axios from 0.18.0 to 0.33.0, convention *Conventional Commits*), notes de version et changelog (rubrique « Security Fixes » pour axios 0.33.0), étiquettes dependencies et

javascript, commandes à écrire en commentaire (@dependabot rebase, @dependabot ignore <dépendance> major version…). Seuls package.json et package-lock.json sont modifiés.

2.3 Le fichier dependabot.yml

Placé dans .github/dependabot.yml sur la branche par défaut, il active les version updates et permet un réglage fin ; alertes et PR de sécurité fonctionnent sans lui. Voici celui de notre dépôt, dans son état actuel :

```yaml
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/"
    schedule:
      interval: "daily"
```

Fichier .github/dependabot.yml de notre dépôt (commit 0f8b178 du 28/09/2026).

Clé	Valeur	Rôle
version	2	Version du format du fichier ; toujours 2.
updates	liste	Un bloc par écosystème et par dossier à surveiller ; ici un seul.
package-ecosystem	"npm"	**Quoi** : le gestionnaire de paquets, donc package.json et package-lock.json.
directory	"/"	**Où** : le dossier qui contient le manifeste, ici la racine du dépôt.
schedule.interval	"daily"	**Quand** : une vérification par jour (autres valeurs : weekly, monthly…).

Tableau 3 : le fichier, clé par clé.

Tout ce qui n'est pas écrit prend sa valeur par défaut : 5 PR de version ouvertes au maximum, aucun regroupement de paquets, aucune exclusion, 3 jours d'attente avant de proposer une nouvelle version, PR ouvertes vers la branche par défaut. C'est ce qu'on observe dans le dépôt : exactement 5 PR de version restent ouvertes (n° 4 à 8).

3. Dependabot dans un pipeline CI/CD

3.1 Détecter, proposer, vérifier, décider

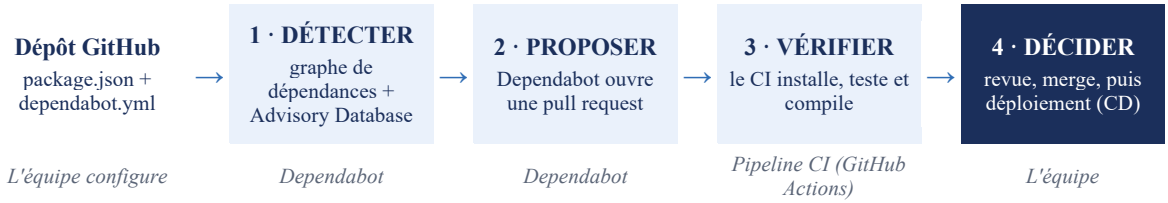


Figure 1 : la place de Dependabot dans le pipeline.

Dependabot n'exécute, ni tests, ni build et ne déploie rien : son travail s'arrête à l'ouverture de la PR. Comme toute PR, celle-ci déclenche le pipeline d'intégration continue de notre dépôt (.github/workflows/ci.yml, un workflow GitHub Actions lancé sur pull_request et push) qui installe les versions exactes du lockfile (npm ci), lance les tests (npm test) et compile (npm run build). L'équipe ne fusionne que si le pipeline est vert ; le déploiement continu (CD) prend le relais.

Ce que notre dépôt a montré. Le 28 septembre, les PR groupées de Dependabot passaient le CI. Après la suppression des groups, il ne restait pour vite que la PR de sécurité n° 10, qui le monte seul de la version 5 à la 8 : le CI l'a marquée en rouge dès l'installation, sur un conflit de « dépendances pairs » (erreur ERESOLVE : @vitejs/plugin-react 4.7.0 n'accepte que vite 4 à 7). La branche main n'étant pas protégée, la PR a pu être fusionnée malgré tout : depuis, le pipeline échoue sur main et sur toutes les PR suivantes. Dependabot a fait son travail ; c'est la règle « on ne fusionne que du vert » qui a manqué.

Quand	Événement	Résultat du CI
28/09 15:34	PR n° 1 (version) : vite 5 → 8 **et** @vitejs/plugin-react 4 → 6, regroupés	✓ **vert**
28/09 15:36	PR n° 10 (sécurité) : vite **seul**, 5.4.21 → 8.3.1	✗ **rouge : ERESOLVE**
28/09 15:37	Push du dependabot.yml simplifié : Dependabot ferme les PR groupées n° 1 et 2	-
28/09 15:45	PR n° 10 fusionnée malgré le CI rouge (branche non protégée)	✗ **main devient rouge**
28/09 15:54	PR n° 9 fusionnée (axios 0.18.0 → 0.33.0) : alertes axios fermées	✗ **rouge (hérité de main)**
30/09 18:19	PR n° 11 fusionnée (axios 0.34.0, lodash 4.18.1) : plus aucune alerte ouverte	✗ **rouge (hérité de main)**

Tableau 4 : chronologie du dépôt (heure de Paris).

Bonnes pratiques. Protéger la branche principale (« Require status checks to pass before merging », avec le check *Tests et build* obligatoire) : une PR rouge ne peut alors plus être fusionnée ; regrouper les paquets liés (groups) ; savoir que les workflows déclenchés par Dependabot reçoivent un GITHUB_TOKEN en lecture seule, sans accès aux secrets GitHub Actions ; réserver la fusion automatique (dependabot/fetch-metadata puis gh pr merge --auto) aux mises à jour mineures et correctives, avec de bons tests.

3.2 Limites et points de vigilance

- **Le bruit.** Pour 15 dépendances déclarées, Dependabot a ouvert 11 PR en trois jours ; trop de PR finissent par ne plus être lues. Remèdes : groups, rythme hebdomadaire, open-pull-requests-limit, ignore.
- **Des mises à jour cassantes, même de sécurité** (vite 5 → 8 dans la PR n° 10) : Dependabot ne garantit pas la compatibilité ; seul le CI la vérifie.
- **Des alertes pas toujours pertinentes.** La détection compare des numéros de version, pas l'usage réel : plusieurs failles d'axios visent son adaptateur HTTP pour Node.js, alors que notre application l'utilise dans le navigateur. À l'équipe de trier, voire de fermer l'alerte avec une justification (*dismiss*).
- **Une couverture partielle.** Un script chargé depuis un CDN, du code copié-collé ou une faille pas encore publiée (*zero-day*) échappent à l'analyse. Sans version corrigée disponible, il y a une alerte mais pas de PR.
- **Nouvelle version ne veut pas dire version sûre.** Le 31 mars 2026, des attaquants ont volé les identifiants npm du mainteneur principal d'axios : les versions 1.14.1 et 0.30.4, publiées avec une dépendance malveillante (plain-crypto-js) qui installait un cheval de Troie, sont restées en ligne environ 3 heures. Depuis le 14 juillet 2026, Dependabot attend par défaut 3 jours avant de proposer une nouvelle version ; les correctifs de sécurité restent immédiats.

Conclusion

Dependabot automatise une tâche que personne ne fait bien à la main : surveiller en continu des centaines de dépendances et proposer leur mise à jour. Il fonctionne selon deux modes complémentaires, sécurité (réactif, sans configuration) et versions (planifié, via dependabot.yml), et se règle pour limiter le bruit et le risque (groups, ignore, open-pull-requests-limit, cooldown). Mais il détecte et propose sans rien tester ni fusionner seul : dans notre dépôt, ses PR ont fermé les 37 alertes, mais une PR de sécurité fusionnée malgré un CI rouge a suffi à casser la branche principale. Sa valeur dépend donc du pipeline CI qui vérifie chaque PR, d'une branche protégée et de l'équipe qui décide. Propre à GitHub, il ne remplace ni l'analyse du code de l'application ni les tests : c'est le premier maillon, gratuit et simple à mettre en place, d'une démarche plus large de sécurité de la chaîne d'approvisionnement logicielle.

Sources

Consultées entre le 25 septembre et le 7 octobre 2026.

- GitHub Docs : « Dependabot security updates », « Dependabot version updates », « About Dependabot alerts », « Dependabot options reference », « Automating Dependabot with GitHub Actions » (docs.github.com/en/code-security).
- GitHub Changelog, 14/07/2026 : « Dependabot version updates introduce default package cooldown ».
- GitHub Advisory Database : avis concernant axios, dont GHSA-jr5f-v2jv-69x6 (CVE-2025-27152) et GHSA-42xw-2xvc-qx8m (CVE-2019-10742).
- axios, « Post Mortem: axios npm supply chain compromise » (github.com/axios/axios/issues/10636), 2026.
- U.S. House Committee on Oversight and Government Reform, « The Equifax Data Breach », décembre 2018.
- Help Net Security, « GitHub introduces Dependabot-powered automated security fixes », 28/05/2019.
- Dépôt de démonstration github.com/camanalyste-hub/demo-dependabotv1 : alertes n° 1 à 37, PR n° 1 à 11, journaux « Dependabot Updates » et « CI » du 28 au 30/09/2026.